```
// ================================================
// ANÁLISIS COMPLETO PARA COLECCIONES PEQUEÑAS (≤100 documentos)
// VERSIÓN MEJORADA - Precisión en mediciones de latencia
// ================================================

// Configuración global MEJORADA
var CONFIG = {
    ITERACIONES: 5000,         // Aumentado para mejor estadística
    WARMUP: 100,               // Aumentado para estabilización
    SHOW_DETAILS: true,
    EXPORT_CSV: false,
    CLEAR_CACHE: false         // Opción para limpiar cache entre pruebas
};

// Función para limpiar cachés (si es posible)
function limpiarCache() {
    try {
        // Limpiar caché de MongoDB
        db.runCommand({clearCursors: 1});
        // Forzar garbage collection en JavaScript
        if (global.gc) global.gc();
    } catch(e) {
        // Ignorar si no está disponible
    }
}

// Función MEJORADA para cálculo de percentiles
function calcularPercentiles(tiempos, percentiles) {
    var resultados = {};
    var n = tiempos.length;

    var tiemposOrdenados = tiempos.slice().sort((a, b) => a - b);

    percentiles.forEach(p => {
        var pos = (p / 100) * (n - 1);
        var posBaja = Math.floor(pos);
        var posAlta = Math.ceil(pos);

        if (posBaja === posAlta) {
            resultados[p] = tiemposOrdenados[posBaja];
        } else {
            // Interpolación lineal para mayor precisión
            var pesoAlto = pos - posBaja;
            var pesoBajo = posAlta - pos;
            resultados[p] = (tiemposOrdenados[posBaja] * pesoBajo) + (tiemposOrdenados[posAlta] * pesoAlto);
        }
    });

    return resultados;
}

// Función MEJORADA para análisis de variabilidad
function analizarVariabilidad(tiempos, promedio) {
    var n = tiempos.length;

    // Calcular desviación estándar
    var sumaCuadrados = tiempos.reduce((sum, t) => sum + Math.pow(t - promedio, 2), 0);
    var desviacion = Math.sqrt(sumaCuadrados / n);

    // Coeficiente de variación
    var cv = (desviacion / promedio) * 100;

    // Rango intercuartílico (IQR) para detectar outliers
    var tiemposOrdenados = tiempos.slice().sort((a,b) => a - b);
    var q1 = tiemposOrdenados[Math.floor(n * 0.25)];
    var q3 = tiemposOrdenados[Math.floor(n * 0.75)];
    var iqr = q3 - q1;
    var limiteInferior = q1 - 1.5 * iqr;
    var limiteSuperior = q3 + 1.5 * iqr;
    var outliers = tiempos.filter(t => t < limiteInferior || t > limiteSuperior);

    return {
        cv: cv.toFixed(2),
        desviacion: desviacion.toFixed(4),
```

```
        iqr: iqr.toFixed(4),
        outliers: outliers.length,
        limiteInferior: limiteInferior,
        limiteSuperior: limiteSuperior,
        estabilidad: cv < 15 ? "alta" : cv < 40 ? "moderada" : "alta variabilidad"
    };
}

// Función mejorada para analizar estructura de explain - VERSIÓN CORREGIDA
function analizarEstructuraExplain(explain) {
    var stats = explain.executionStats;

    // Obtener el stage principal - CORREGIDO
    var stage;
    if (stats.executionStages) {
        stage = stats.executionStages;
    } else if (stats.allPlansExecution && stats.allPlansExecution.length > 0) {
        // Para algunas consultas complejas
        stage = stats.allPlansExecution[0].executionStages;
    } else {
        // Si no encontramos executionStages, intentamos obtener del primer elemento de stages
        stage = { stage: "DESCONOCIDO", indexName: "desconocido" };
    }

    // Información base
    var result = {
        stage: stage.stage || "DESCONOCIDO",
        nReturned: stats.nReturned || 0,
        totalDocsExamined: stats.totalDocsExamined || 0,
        totalKeysExamined: stats.totalKeysExamined || 0,
        executionTimeMillis: stats.executionTimeMillis || 0
    };

    // Función auxiliar para buscar recursivamente un stage específico
    function buscarStage(stageActual, tipos) {
        if (tipos.includes(stageActual.stage)) {
            return stageActual;
        }
        if (stageActual.inputStage) {
            return buscarStage(stageActual.inputStage, tipos);
        }
        if (stageActual.shards) {
            for (var shard of stageActual.shards) {
                var encontrado = buscarStage(shard.executionStages, tipos);
                if (encontrado) return encontrado;
            }
        }
        if (stageActual.innerStage) {
            return buscarStage(stageActual.innerStage, tipos);
        }
        if (stageActual.outerStage) {
            return buscarStage(stageActual.outerStage, tipos);
        }
        if (stageActual.stages) {
            for (var s of stageActual.stages) {
                var encontrado = buscarStage(s, tipos);
                if (encontrado) return encontrado;
            }
        }
        return null;
    }

    // Detectar el tipo de escaneo principal
    var scanStage = buscarStage(stage, ["IXSCAN", "EXPRESS_IXSCAN", "COLLSCAN", "TEXT", "FETCH"]);

    if (!scanStage) {
        // Si no encontramos un stage de escaneo, usamos el stage principal
        result.tipoEscaneo = stage.stage;
        result.indiceUsado = "desconocido";
        result.inputStage = stage.inputStage ? stage.inputStage.stage : null;
    }
    else if (scanStage.stage === "EXPRESS_IXSCAN" || scanStage.stage === "IXSCAN") {
        // Escaneo directo por índice
        result.tipoEscaneo = "IXSCAN";
```

```
        result.indiceUsado = scanStage.indexName || "desconocido";
        result.inputStage = null;
    }
    else if (scanStage.stage === "FETCH") {
        // FETCH con posible IXSCAN como inputStage
        result.tipoEscaneo = "FETCH";
        if (scanStage.inputStage &&
            (scanStage.inputStage.stage === "IXSCAN" || scanStage.inputStage.stage === "EXPRESS_IXSCAN")) {
            result.tipoEscaneo = "FETCH con IXSCAN";
            result.indiceUsado = scanStage.inputStage.indexName || "desconocido";
        } else {
            result.indiceUsado = "ninguno";
        }
        result.inputStage = scanStage.inputStage ? scanStage.inputStage.stage : null;
    }
    else if (scanStage.stage === "COLLSCAN") {
        // Escaneo completo
        result.tipoEscaneo = "COLLSCAN";
        result.indiceUsado = "ninguno";
        result.inputStage = null;
    }
    else if (scanStage.stage === "TEXT") {
        // Búsqueda de texto
        result.tipoEscaneo = "TEXT";
        result.indiceUsado = scanStage.indexName || "desconocido";
        result.inputStage = null;
    }
    else {
        // Otro tipo (SHARDING, SORT, etc.)
        result.tipoEscaneo = scanStage.stage;
        result.indiceUsado = "desconocido";
        result.inputStage = scanStage.inputStage ? scanStage.inputStage.stage : null;
    }

    // Si todavía no tenemos tipo de escaneo, usar el stage principal
    if (!result.tipoEscaneo) {
        result.tipoEscaneo = stage.stage || "DESCONOCIDO";
    }

    return result;
}

// Función principal de análisis MEJORADA
function analizarConsultaProfesional(nombre, consulta, proyeccion = {}) {
    print("\n" + "=".repeat(90));
    print(`🔍 ANÁLISIS: ${nombre}`);
    print("=".repeat(90));

    // ================================================
    // 1. ANÁLISIS DE PLAN DE EJECUCIÓN
    // ================================================
    print("\n📊 1. PLAN DE EJECUCIÓN:");

    try {
        var explain = db.PlantaB.find(consulta, proyeccion).explain("executionStats");
        var analysis = analizarEstructuraExplain(explain);
        var stats = explain.executionStats;

        // Información básica
        print(`   • Stage: ${analysis.stage}`);
        print(`   • Tipo de escaneo: ${analysis.tipoEscaneo}`);
        print(`   • Índice usado: ${analysis.indiceUsado}`);
        print(`   • Documentos devueltos: ${analysis.nReturned}`);
        print(`   • Documentos examinados: ${analysis.totalDocsExamined}`);
        print(`   • Claves de índice examinadas: ${analysis.totalKeysExamined}`);
        print(`   • Tiempo ejecución MongoDB: ${analysis.executionTimeMillis} ms`);

        // Mostrar estructura si se solicita
        if (CONFIG.SHOW_DETAILS && analysis.inputStage) {
            print(`   • Input stage: ${analysis.inputStage}`);
        }

    } catch (e) {
        print(`   ❌ Error al analizar explain: ${e.message}`);
```

```
    return null;
}

// ================================================
// 2. MÉTRICA DE EFICIENCIA/SELECTIVIDAD
// ================================================
print("\n📈 2. EFICIENCIA / SELECTIVIDAD:");

var eficiencia = (analysis.nReturned / analysis.totalDocsExamined * 100).toFixed(4);
var selectividad = analysis.totalDocsExamined > 0 ?
    (analysis.nReturned / analysis.totalDocsExamined).toFixed(4) : 0;

print(`   • Eficiencia: ${eficiencia}%`);
print(`   • Selectividad: ${selectividad} (${analysis.nReturned}/${analysis.totalDocsExamined})`);

// Diagnóstico mejorado
var diagnostico = "";
if (analysis.tipoEscaneo.includes("IXSCAN") || analysis.tipoEscaneo.includes("FETCH con IXSCAN")) {
    if (eficiencia == 100) {
        diagnostico = "✅ ÓPTIMA (índice funciona perfectamente)";
    } else if (eficiencia > 50) {
        diagnostico = "✅ BUENA (índice selectivo)";
    } else {
        diagnostico = "⚠️  ÍNDICE POCO SELECTIVO";
    }
} else if (analysis.tipoEscaneo === "COLLSCAN") {
    if (eficiencia == 100) {
        diagnostico = "⚠️  COLLSCAN COMPLETO (normal para consulta sin filtros)";
    } else if (eficiencia < 10) {
        diagnostico = "❌ CRÍTICA (COLLSCAN para pocos resultados - necesita índice)";
    } else {
        diagnostico = "⚠️  COLLSCAN (considerar índice para mejorar)";
    }
}

print(`   • Diagnóstico: ${diagnostico}`);

// ================================================
// 3. BENCHMARK DE LATENCIA (VERSIÓN MEJORADA)
// ================================================
print("\n⏱️  3. LATENCIA / TIEMPO DE RESPUESTA (MEDICIÓN MEJORADA):");

// Warm-up MEJORADO
print(`   • Fase warm-up: ${CONFIG.WARMUP} iteraciones`);
for (var i = 0; i < CONFIG.WARMUP; i++) {
    var cursor = db.PlantaB.find(consulta, proyeccion);
    cursor.itcount(); // Forzar ejecución completa
}

if (CONFIG.CLEAR_CACHE) {
    print(`   • Limpiando cachés...`);
    limpiarCache();
}

// Mediciones MEJORADAS
var tiempos = [];
var tiemposMongo = []; // Para comparar con executionTimeMillis

for (var i = 0; i < CONFIG.ITERACIONES; i++) {
    // Medir tiempo del cliente
    var start = Date.now();
    var cursor = db.PlantaB.find(consulta, proyeccion);
    var count = cursor.itcount();
    var end = Date.now();
    tiempos.push(end - start);

    // Cada 100 iteraciones, verificar consistencia con explain
    if (i % 100 === 0) {
        var explainTemp = db.PlantaB.find(consulta, proyeccion).explain("executionStats");
        tiemposMongo.push(explainTemp.executionStats.executionTimeMillis);
    }
}
```

```
    // Estadísticas MEJORADAS
    var suma = tiempos.reduce((a, b) => a + b, 0);
    var promedio = suma / CONFIG.ITERACIONES;
    var min = Math.min(...tiempos);
    var max = Math.max(...tiempos);

    // Percentiles con interpolación
    var percentiles = [50, 75, 90, 95, 99, 99.9];
    var percentilValues = calcularPercentiles(tiempos, percentiles);

    // Análisis de variabilidad MEJORADO
    var variabilidad = analizarVariabilidad(tiempos, promedio);

    print(`   • Iteraciones: ${CONFIG.ITERACIONES}`);
    print(`   • Promedio: ${promedio.toFixed(4)} ms`);
    print(`   • Mínimo: ${min} ms, Máximo: ${max} ms`);
    print(`   • Rango: ${(max - min).toFixed(2)} ms`);
    print(`   • Coeficiente de variación: ${variabilidad.cv}% (${variabilidad.estabilidad})`);
    print(`   • IQR (P75-P25): ${variabilidad.iqr} ms`);
    print(`   • Outliers detectados: ${variabilidad.outliers}
(${((variabilidad.outliers/CONFIG.ITERACIONES)*100).toFixed(2)}%)`);

    percentiles.forEach(p => {
        if (p >= 50) {
            print(`   • P${p}: ${percentilValues[p].toFixed(4)} ms`);
        }
    });

    // Comparación con MongoDB internal time
    if (tiemposMongo.length > 0) {
        var avgMongo = tiemposMongo.reduce((a,b) => a + b, 0) / tiemposMongo.length;
        print(`   • Tiempo interno MongoDB (avg): ${avgMongo.toFixed(4)} ms`);
        print(`   • Diferencia cliente/servidor: ${Math.abs(promedio - avgMongo).toFixed(4)} ms
(overhead)`);
    }

    // NUEVA SECCIÓN: Análisis de estabilidad
    print("\n📊 3b. ANÁLISIS DE ESTABILIDAD:");
    // Dividir mediciones en segmentos para ver tendencia
    var segmentos = 5;
    var segmentSize = Math.floor(CONFIG.ITERACIONES / segmentos);
    var promediosSegmentos = [];

    for (var s = 0; s < segmentos; s++) {
        var inicio = s * segmentSize;
        var fin = inicio + segmentSize;
        var segmento = tiempos.slice(inicio, fin);
        var promedioSeg = segmento.reduce((a,b) => a + b, 0) / segmento.length;
        promediosSegmentos.push(promedioSeg);

        if (CONFIG.SHOW_DETAILS) {
            print(`   • Segmento ${s+1}: ${promedioSeg.toFixed(4)} ms (iter ${inicio}-${fin-1})`);
        }
    }

    // Calcular deriva (drift)
    var primerSegmento = promediosSegmentos[0];
    var ultimoSegmento = promediosSegmentos[segmentos-1];
    var deriva = ((ultimoSegmento - primerSegmento) / primerSegmento * 100).toFixed(2);
    var tendencia = deriva > 5 ? "🔴 Aumentando" : deriva < -5 ? "🟠 Disminuyendo" : "🟢 Estable";

    print(`   • Deriva temporal: ${deriva}% ${tendencia}`);

    // ================================================
    // 4. COSTO DE ÍNDICES
    // ================================================
    print("\n💰 4. COSTO DE ÍNDICES / SOBRECARGA:");

    var costoIndice = 0;
    var costoDescripcion = "";

    if (analysis.tipoEscaneo.includes("IXSCAN")) {
        costoIndice = analysis.totalKeysExamined;
        costoDescripcion = "IXSCAN (búsqueda por índice)";
```

```
        print(`    • Tipo: ${costoDescripcion}`);
        print(`    • Claves examinadas: ${costoIndice}`);
        print(`    • Costo por documento: ${(costoIndice / (analysis.nReturned || 1)).toFixed(2)}
claves/doc`);

        if (analysis.totalKeysExamined === analysis.nReturned) {
            print(`    • Eficiencia índice: 100% (índice altamente selectivo)`);
        } else if (analysis.totalKeysExamined > analysis.nReturned) {
            print(`    • Eficiencia índice: ${((analysis.nReturned / analysis.totalKeysExamined) *
100).toFixed(2)}%`);
        }
    }
    else if (analysis.tipoEscaneo.includes("FETCH con IXSCAN")) {
        costoIndice = analysis.totalKeysExamined;
        costoDescripcion = "FETCH con IXSCAN (índice + recuperación)";
        print(`    • Tipo: ${costoDescripcion}`);
        print(`    • Claves examinadas: ${costoIndice}`);
        print(`    • Documentos recuperados: ${analysis.nReturned}`);
        print(`    • Costo total: ${costoIndice} claves + ${analysis.nReturned} documentos`);
    }
    else if (analysis.tipoEscaneo === "COLLSCAN") {
        costoIndice = analysis.totalDocsExamined;
        costoDescripcion = "COLLSCAN (escaneo completo)";
        print(`    • Tipo: ${costoDescripcion}`);
        print(`    • Documentos examinados: ${costoIndice}`);
        print(`    • Costo: Escaneo de ${costoIndice} documentos`);

        if (analysis.totalDocsExamined > 10 && analysis.nReturned < analysis.totalDocsExamined * 0.1) {
            print(`    • ⚠️  Recomendación: Considerar crear índice (solo
${analysis.nReturned}/${analysis.totalDocsExamined} documentos útiles)`);
        }
    }
    else {
        costoIndice = analysis.totalDocsExamined;
        costoDescripcion = analysis.tipoEscaneo;
        print(`    • Tipo: ${costoDescripcion} (escaneo especial)`);
        print(`    • Unidades examinadas: ${costoIndice}`);
    }

    // ================================================
    // 5. RESUMEN EJECUTIVO (MEJORADO)
    // ================================================
    print("\n📋 5. RESUMEN EJECUTIVO:");

    var resumen = {
        "Consulta": nombre,
        "Stage": analysis.stage,
        "TipoEscaneo": analysis.tipoEscaneo,
        "Índice": analysis.indiceUsado,
        "Devueltos": analysis.nReturned,
        "Examinados": analysis.totalDocsExamined,
        "ClavesExaminadas": analysis.totalKeysExamined,
        "Eficiencia": eficiencia + "%",
        "LatenciaPromedio": promedio.toFixed(4) + " ms",
        "LatenciaP50": percentilValues[50].toFixed(4) + " ms",
        "LatenciaP95": percentilValues[95].toFixed(4) + " ms",
        "LatenciaP99": percentilValues[99].toFixed(4) + " ms",
        "VariabilidadCV": variabilidad.cv + "%",
        "Estabilidad": variabilidad.estabilidad,
        "Outliers": variabilidad.outliers,
        "Deriva": deriva + "%",
        "CostoOperacion": costoDescripcion,
        "CostoUnidades": costoIndice,
        "CostoPorDoc": (costoIndice / (analysis.nReturned || 1)).toFixed(2),
        "Diagnostico": diagnostico,
        "Iteraciones": CONFIG.ITERACIONES
    };

    // Mostrar resumen en tabla compacta
    print("    " + "-".repeat(70));
    print(`    • Escaneo: ${resumen.TipoEscaneo} (${resumen.Índice})`);
    print(`    • Eficiencia: ${resumen.Eficiencia} (${resumen.Devueltos}/${resumen.Examinados})`);
    print(`    • Latencia: ${resumen.LatenciaPromedio} (P95: ${resumen.LatenciaP95})`);
    print(`    • Variabilidad: ${resumen.VariabilidadCV} (${resumen.Estabilidad})`);
```

```
    print(`    • Estabilidad: Deriva ${resumen.Deriva}, Outliers: ${resumen.Outliers}`);
    print(`    • Costo: ${resumen.CostoUnidades} unidades (${resumen.CostoPorDoc}/doc)`);
    print("   " + "-".repeat(70));

    // ================================================
    // 6. VISUALIZACIÓN MEJORADA
    // ================================================
    if (CONFIG.SHOW_DETAILS) {
        print("\n📊 DISTRIBUCIÓN DE TIEMPOS:");

        // Crear histograma con bins de 0.1ms
        var histograma = {};
        var maxFreq = 0;

        tiempos.forEach(t => {
            var bin = Math.floor(t * 10) / 10; // Agrupar por 0.1ms
            histograma[bin] = (histograma[bin] || 0) + 1;
            if (histograma[bin] > maxFreq) maxFreq = histograma[bin];
        });

        // Ordenar y mostrar
        var sortedBins = Object.keys(histograma).map(Number).sort((a, b) => a - b);
        var maxBars = 40;

        sortedBins.forEach(bin => {
            var count = histograma[bin];
            var porcentaje = (count / CONFIG.ITERACIONES * 100).toFixed(1);
            var barras = Math.round((count / maxFreq) * maxBars);
            var barra = "".repeat(barras);

            // Color basado en percentil
            var colorCode = "";
            if (bin <= percentilValues[50]) colorCode = "🟢"; // P50 o menos
            else if (bin <= percentilValues[90]) colorCode = "🟡"; // P50-P90
            else if (bin <= percentilValues[95]) colorCode = "🟠"; // P90-P95
            else colorCode = "🔴"; // P95+

            print(`   ${colorCode} ${bin.toFixed(1)} ms: ${barra} ${porcentaje}% (${count})`);
        });

        // Resumen estadístico
        print("\n   📈 ESTADÍSTICAS DE DISTRIBUCIÓN:");
        print(`     • Media: ${promedio.toFixed(4)} ms`);
        print(`     • Mediana (P50): ${percentilValues[50].toFixed(4)} ms`);

        // Calcular moda
        var modaBin = Object.keys(histograma).reduce((a, b) => histograma[a] > histograma[b] ? a : b);
        print(`     • Moda: ${parseFloat(modaBin).toFixed(1)} ms`);
        print(`     • Asimetría: ${(promedio > percentilValues[50] ? "derecha" : promedio <
percentilValues[50] ? "izquierda" : "simétrica")}`);
        print(`     • Outliers: ${variabilidad.outliers}
(${((variabilidad.outliers/CONFIG.ITERACIONES)*100).toFixed(2)}%)`);
    }

    return resumen;
}

// ================================================
// EJECUCIÓN DE PRUEBA RÁPIDA (MEJORADA)
// ================================================
function pruebaRapida() {
    print("=".repeat(100));
    print("🚀 PRUEBA RÁPIDA MEJORADA - ANÁLISIS CON ÍNDICES ESPECÍFICOS");
    print("=".repeat(100));

    print("\n📝 CONFIGURACIÓN ACTUAL:");
    print(`    • Iteraciones: ${CONFIG.ITERACIONES}`);
    print(`    • Warm-up: ${CONFIG.WARMUP}`);
    print(`    • Limpiar caché: ${CONFIG.CLEAR_CACHE ? "SÍ" : "NO"}`);
    print(`    • Mostrar detalles: ${CONFIG.SHOW_DETAILS ? "SÍ" : "NO"}`);

    // Verificar que los índices existan antes de ejecutar
    print("\n🔍 VERIFICANDO ÍNDICES DISPONIBLES:");
```

```
var indices = db.PlantaB.getIndexes();
var indicesNombres = indices.map(idx => idx.name);

print("   • Índices encontrados: " + indicesNombres.join(", "));

// ================================================
// 1. PRUEBA ÍNDICE DE TEXTO (búsqueda en PREVIEW)
// ================================================
if (indicesNombres.includes("idx_busqueda_texto_preview")) {
    print("\n" + "=".repeat(70));
    print("1. Probando índice de texto (búsqueda en PREVIEW):");
    var resultado1 = analizarConsultaProfesional(
        "Índice de texto - PREVIEW",
        {
            $text: {
                $search: "Werneria"
            }
        },
        {
            "PREVIEW": 1,
            "score": { $meta: "textScore" },  // Importante para índices de texto
            "_id": 0
        }
    );
} else {
    print("\n❌ Índice de texto 'idx_busqueda_texto_preview' no encontrado");
}

// ================================================
// 2. PRUEBA ÍNDICE COMPUESTO TAXONÓMICO
// ================================================
if (indicesNombres.includes("idx_taxonomico_BTJ")) {
    print("\n" + "=".repeat(70));
    print("\n2. Probando índice compuesto taxonómico (family + genus):");
    var resultado2 = analizarConsultaProfesional(
        "Índice compuesto taxonómico",
        {
            "TAXONOMICO.family": "Ericaceae",
            "TAXONOMICO.genus": "Disterigma"
        },
        { "TAXONOMICO": 1, "PREVIEW": 1, "_id": 0 }
    );
} else {
    print("\n❌ Índice 'idx_taxonomico_BTJ' no encontrado");
}

// ================================================
// 3. PRUEBA ÍNDICE COMPUESTO FITOQUIMICO
// ================================================
if (indicesNombres.includes("idx_fitoquimico_type_value")) {
    print("\n" + "=".repeat(70));
    print("\n3. Probando índice compuesto fitoquímico (measurementType + measurementValue):");
    var resultado3 = analizarConsultaProfesional(
        "Índice compuesto fitoquímico",
        {
            "FITOQUIMICO.measurementType": "Antraquinonas",
            "FITOQUIMICO.measurementValue": { $gt: "2.1" }
        },
        { "FITOQUIMICO": 1, "PREVIEW.scientificName": 1, "_id": 0 }
    );
} else {
    print("\n❌ Índice 'idx_fitoquimico_type_value' no encontrado");
}

// ================================================
// 4. PRUEBA ÍNDICE FISICOQUIMICO
// ================================================
if (indicesNombres.includes("idx_fisicoquimico_method")) {
    print("\n" + "=".repeat(70));
    print("\n4. Probando índice fisicoquímico (measurementMethod):");
    var resultado4 = analizarConsultaProfesional(
        "Índice fisicoquímico",
        {
            "FISICOQUIMICO.measurementMethod": "Incineración"
```

```
            },
            { "FISICOQUIMICO": 1, "PREVIEW.scientificName": 1, "_id": 0 }
        );
    } else {
        print("\n❌ Índice 'idx_fisicoquimico_method' no encontrado");
    }

    // ================================================
    // 5. PRUEBA CONSULTA COMPLEJA MÚLTIPLES ÍNDICES
    // ================================================
    print("\n" + "=".repeat(70));
    print("\n5. Probando consulta compleja (combinando múltiples criterios):");
    var resultado5 = analizarConsultaProfesional(
        "Consulta compleja múltiple",
        {
            $and: [
                { "TAXONOMICO.family": "Amaryllidaceae" },
                { "FITOQUIMICO.measurementType": "Alcaloides" },
                { "FISICOQUIMICO.measurementMethod": "Extracción acuosa y evaporación" }
            ]
        },
        {
            "TAXONOMICO": 1,
            "FITOQUIMICO": 1,
            "FISICOQUIMICO": 1,
            "PREVIEW": 1,
            "_id": 0
        }
    );

    // ================================================
    // 6. PRUEBA CONSULTA SIN ÍNDICE ESPECÍFICO (COLLSCAN)
    // ================================================
    print("\n" + "=".repeat(70));
    print("\n6. Probando consulta sin índice específico (para comparación):");
    var resultado6 = analizarConsultaProfesional(
        "COLLSCAN completo",
        {},
        {}
    );

    return [resultado1, resultado2, resultado3, resultado4, resultado5, resultado6];
}

// ================================================
// FUNCIÓN PARA AJUSTAR CONFIGURACIÓN
// ================================================
function ajustarConfiguracion(nuevoConfig) {
    print("\n⚙️  AJUSTANDO CONFIGURACIÓN:");
    for (var key in nuevoConfig) {
        if (CONFIG.hasOwnProperty(key)) {
            print(`   • ${key}: ${CONFIG[key]} → ${nuevoConfig[key]}`);
            CONFIG[key] = nuevoConfig[key];
        }
    }
    return CONFIG;
}

// ================================================
// MENÚ PRINCIPAL
// ================================================
function menuPrincipal() {
    print("\n🔧 Ejecutando prueba rápida por defecto...");
    return pruebaRapida();
}

// ================================================
// EJECUCIÓN AUTOMÁTICA
// ================================================
print("\n🚀 INICIANDO ANALIZADOR DE CONSULTAS MONGODB...");

// Ejecutar análisis
var resultados = menuPrincipal();
```

```
// Resumen final
print("\n" + "=".repeat(100));
print("📋 RESUMEN FINAL DE TODAS LAS PRUEBAS");
print("=".repeat(100));

if (resultados && resultados.length > 0) {
    resultados.forEach((res, idx) => {
        if (res) {
            print(`\n${idx+1}. ${res.Consulta}:`);
            print(`    • Tipo: ${res.TipoEscaneo}`);
            print(`    • Latencia: ${res.LatenciaPromedio} (P95: ${res.LatenciaP95})`);
            print(`    • Variabilidad: ${res.VariabilidadCV} (${res.Estabilidad})`);
            print(`    • Outliers: ${res.Outliers}`);
        }
    });

    // Comparación
    print("\n📊 COMPARACIÓN ENTRE CONSULTAS:");
    var mejorLatencia = resultados.reduce((prev, curr) =>
        parseFloat(prev.LatenciaPromedio) < parseFloat(curr.LatenciaPromedio) ? prev : curr
    );
    var mejorEstabilidad = resultados.reduce((prev, curr) =>
        parseFloat(prev.VariabilidadCV) < parseFloat(curr.VariabilidadCV) ? prev : curr
    );

    print(`    • Más rápida: ${mejorLatencia.Consulta} (${mejorLatencia.LatenciaPromedio})`);
    print(`    • Más estable: ${mejorEstabilidad.Consulta} (${mejorEstabilidad.VariabilidadCV}%)`);
}

print("\n✅ ANÁLISIS COMPLETADO");
```